

Raport Laborator: Migrare la ORM și Connection Pooling

1. Comparație cod: Înainte (JDBC) vs. După (ORM)

Înainte (JDBC pur): Pentru fiecare entitate exista o clasă separată de tip Repository (`StudentRepository`, `ProfessorRepository` etc.). Execuția implica scrierea manuală a interogărilor SQL sub formă de String, setarea manuală a parametrilor în `PreparedStatement` și extragerea repetitivă a datelor din `ResultSet`. **După (Hibernate ORM):** Toate clasele de Repository specifice au fost șterse. Interacțiunea cu baza de date a fost centralizată într-o singură clasă `HibernateRepository<T>` generică. **Analiza reducerii liniilor de cod:** Am eliminat complet sute de linii de cod repetitiv (boilerplate). Maparea datelor se face acum automat prin adnotări (`@Entity`, `@OneToMany`, `@ManyToOne`), iar metodele CRUD folosesc direct API-ul Hibernate (ex: `session.persist()`, `session.get()`), scurtând logica de salvare sau căutare de la 15-20 de linii la doar 2-3 linii.

2. Analiza Performanței (Sarcina A - Connection Pooling)

Pentru a evalua overhead-ul creării de conexiuni, am testat obținerea a 100 de conexiuni consecutive la baza de date PostgreSQL.

- **Fără pooling (JDBC DriverManager):** * Timp total: ~621 ms
 - Timp mediu per conexiune: 6.21 ms
- **Cu pooling (HikariCP - Pool size 10):**
 - Timp total: ~4 ms
 - Timp mediu per împrumut: 0.04 ms

Interpretare: Utilizarea HikariCP a făcut aplicația de peste **150 de ori mai rapidă** în gestionarea resurselor. Fără pooling, fiecare cerere a necesitat handshake de rețea, autentificare și alocare de memorie pe server. Cu pooling, conexiunile sunt menținute deschise în fundal, iar aplicația le "împrumută" instantaneu.

3. Detectarea Scurgerilor de Conexiuni (Sarcina B - Connection Leaks)

Am simulat un scenariu în care pool-ul este limitat la 10 conexiuni, iar o buclă a solicitat 11 conexiuni succesive **fără** a apela metoda `close()`.

- **Comportament observat:** La solicitarea celei de-a 11-a conexiuni, aplicația s-a blocat temporar (așteptând ca o resursă să se elibereze), iar după expirarea timpului de timeout, a aruncat o excepție fatală (epuizarea pool-ului).
- **Remediere:** Problema s-a rezolvat prin implementarea blocului `try-with-resources` (ex: `try (Session session = HibernateUtil.getSessionFactory().openSession())`). Astfel, la

finalizarea blocului, metoda `close()` este apelată automat, iar conexiunea se întoarce în siguranță în pool.

4. Evaluare: Lazy vs. Eager Loading

- **Lazy Loading (Implicit):** La apelul entității `Professor`, Hibernate a executat un singur `SELECT` pentru datele profesorului. Lista de materii nu a fost încărcată. Abia la apelul metodei `getClasses()`, a fost declanșat un al doilea query separat. Avantajul este economia de memorie.
- **Eager Loading (prin `HQL JOIN FETCH`):** Am forțat aducerea tuturor datelor simultan. Hibernate a generat o singură interogare complexă (cu `LEFT JOIN`). Deși inițial query-ul este mai "greu", evită problema N+1 atunci când știm sigur că vom avea nevoie de datele asociate.

5. Concluzii asupra tehnologiei ORM

Avantaje:

- Productivitate enormă și cod curat, orientat strict pe obiecte.
- Mentenabilitate ușoară (schimbarea tipului de bază de date necesită doar modificarea "dialectului" în `hibernate.cfg.xml`, fără a rescrie cod SQL).
- Gestionarea automată a relațiilor complexe și a tranzacțiilor.

Dezavantaje:

- Curbă de învățare mai abruptă comparativ cu SQL clasic.
- Erorile apărute la configurare (de tip *Reflection* sau mapare) pot fi abstractizate și greu de depanat inițial.

Provocări întâmpinate în timpul migrării:

1. **Conflict de nume (Naming Collision):** Clasa mea de domeniu se numea `Class`. Hibernate o confunda cu clasa nativă `java.lang.Class` la procesul de reflection, aruncând excepții de castare (`BasicTypeImpl`). Am rezolvat prin redenumirea entității în `CollegeClass`.
2. **Pozitionarea Adnotărilor:** Plasarea inițială a adnotării `@Id` deasupra metodei `getId()` (care returna un generic `ID`) a determinat Hibernate să încerce maparea spre tipul `Array` în PostgreSQL. Problema a fost rezolvată mutând adnotarea `@Id` direct pe câmpul `private Integer id`.